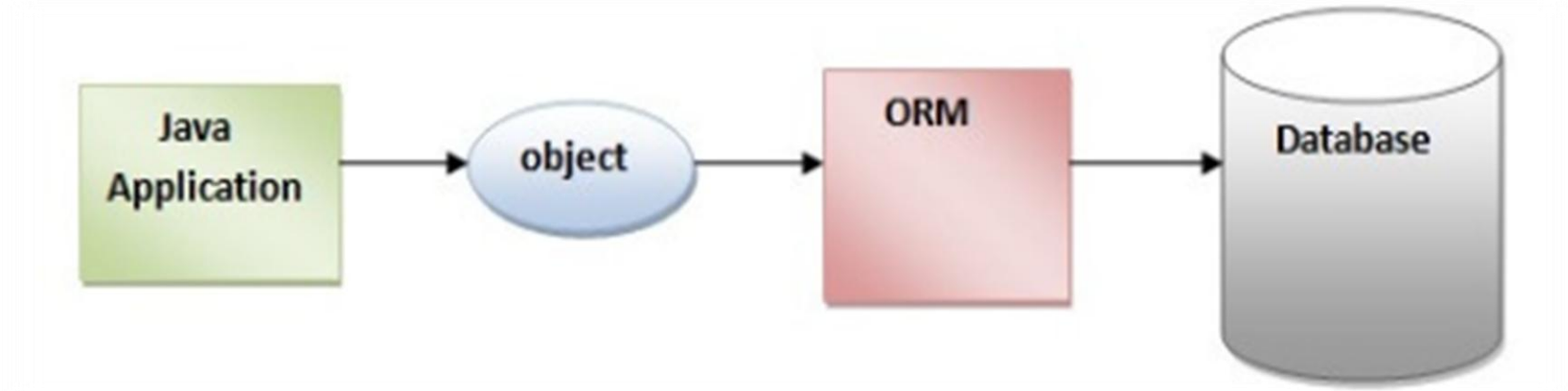
HIBERNATE FRAMEWORK

HIBERNATE FRAMEWORK

- **Hibernate is a Java framework that simplifies the development of Java application to interact with the database.**
- **It is an open source, lightweight framework.**
- **Hibernate is an ORM (Object Relational Mapping) tool.**
- **Hibernate implements the specifications of JPA (Java Persistence API) for data persistence.**

ORM TOOL
(Object Relational Mapping Tool)

- **An ORM tool simplifies the data creation, data manipulation and data access.**
- **It is a programming technique that maps the object to the data stored in the database.**



- **The ORM tool internally uses the JDBC API to interact with the database.**

JPA

- **Java Persistence API (JPA) is a Java specification(collection of classes and interfaces) that provides certain functionality and standard to ORM tools.**

- **The javax.persistence package contains the JPA classes and interfaces.**

- **It facilitates ORM tool to manage relational data in Java applications.**

- **It provides a platform to work directly with objects instead of using SQL statements.**

Advantages of Hibernate Framework

1) Open Source and Lightweight : **Hibernate framework is open source under the LGPL license and lightweight.**

2) Fast Performance : **The performance of hibernate framework is fast because cache is internally used in hibernate framework.**

3) Database Independent Query : Hibrenate supports Hibernate Query Language. HQL is the object-oriented version of SQL. It generates the database independent queries. So you don't need to write database specific queries. Before Hibernate, if database is changed for the project, we need to change the SQL query as well.

4) Automatic Database and Table Creation : Hibernate framework provides the facility to create the tables and database automatically depending on which DB we are using. So there is no need to create tables or database manually.

For creating the a hibernate application, we need to follow the following steps:

1. **Create the Persistent / Entity class.**

2. **Do the mapping for Entity class using @Entity , @Id annotations etc.**

3. **Create the Configuration file(persistence.xml) by specifying database name , username and password of database etc.**

4. **Create the class that stores or retrieves the persistent object , i.e.. Class to perform database operations.**

5. **Run the program.**

ENTITY CLASS

- **Java classes whose objects will be stored in database tables are called persistent classes in Hibernate. They are also called as Entity class.**
-
- **The entity class is a class that represents a table in a database .**
- **To mark any class as Entity class , the class must be annotated with @Entity annotation.**

Entity class must follow the following rules :

1. **A no-arg constructor:** It is recommended to have a NO argument constructor so that hibernate can create the object of the Persistent class.

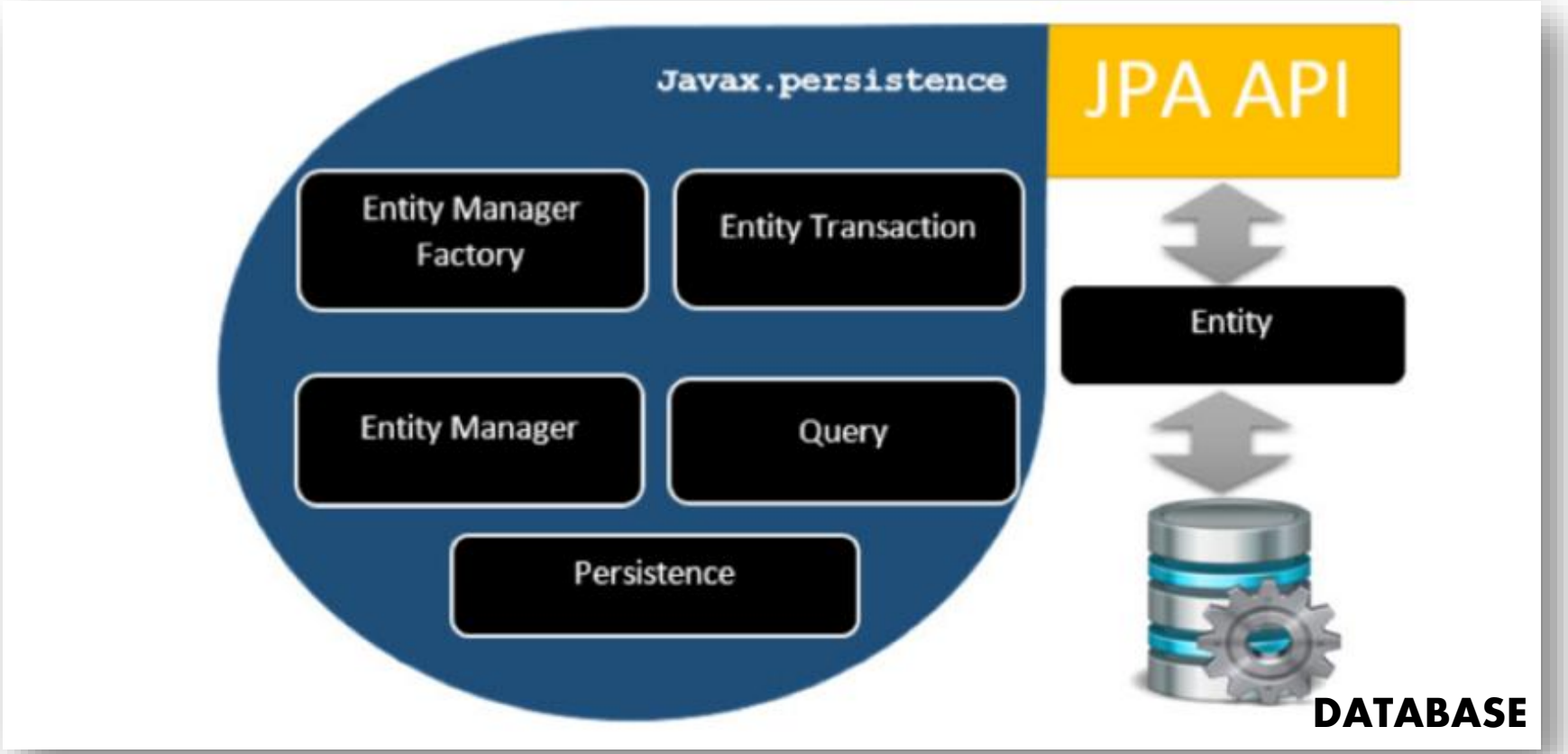
2. **Map the class :** Annotate the persistent class with @Entity annotation to map the class with the table in database.

3. **Primary key :** An attribute must be specified as a Primary key using @Id annotation. This attribute behaves as a primary key in database.

4. **Getter and setter methods:** The Hibernate recognizes the attributes by getter and setter method names by default. So getter and setter methods must be created for all the attributes.

COMPONENTS OF JPA (OR)
CLASS LEVEL ARCHITECHTURE OF JPA

- **The following image shows the class level architecture of JPA. It shows the core classes and interfaces of JPA.**



- **Persistence :** **This class contains static methods , like createEntityManagerFactory(String persistenceUnitName) to obtain EntityManagerFactory instance. It accepts the persistence unit name as input and loads all the properties from the persistence file.**

- **EntityManagerFactory :** **This is a factory class of EntityManager. It consists of createEntityManager() method creates , returns and manages multiple EntityManager instances.**

- **EntityManager :** **is the primary JPA interface. The EntityManager manages the entities and consists of methods to perform database operations such insert , update , delete and retrieve objects from database.**

- **It consists of following methods :**

  - persist(object) : **It is used to store(insert) an entity object into the database.**
  - remove(object) : **It is used to remove an entity object from the database.**
  - merge(object) : **It is used to update the existing object in the database by merging the states of given entity object.**
  - find(entityClassName.class , primaryKeyValue) : **Used to find / search for object based on primary key.**
  - getTransaction() : **creates and return an object of EntityTransaction.**
  - createQuery(String sqlString) : **It accepts a Java Persistence query language (JPQL) query as an input and creates an object of Query for executing JPQL statement.**

- **EntityTransaction :** For each EntityManager, operations are maintained by EntityTransaction. It is used to keep track of transactions performed by the EntityManager. It consists of methods such as :

    - begin() : it is used to specify that transaction has started.
    - commit() : it specifies that the transaction has ended and saves then changes to the database.

- **Query :** The object of Query can be obtained by calling the createQuery() of EntityManager interface. The Query object is used to execute the JPQL / HQL/SQL queries. For executing the queries , it consists of methods such as ,

    - int executeUpdate() : is used to execute the update or delete query.
    - List<E> getResultList() : is used to execute the select query. It returns the results as a List.

ANNOTATIONS

@Entity : Used for declaring any POJO class as an entity(table) for a database. It maps the class with the table in the database.

@Table : Used to change table details, such as name of the table. If you don't use @Table annotation, hibernate will use the class name as the table name by default.

@Id : marks the identifier for the entity. It is Used for declaring a primary key inside our entity class.

@GeneratedValue : Used to automatically generate the primary key values based on certain strategies and we don't need to set the values manually.

- **@Column** : used to specify the details of the column to which a field or property will be mapped. @Column annotation can be used with following attributes :
    - **name** attribute allows to specify the name of the column explicitly.
    - **length** attribute allows to specify the size of the column , particularly for a String value.
    - **nullable** attribute allows to specify whether null values are allowed in the column or not.
    - **unique** attribute allows to specify whether duplicate values are allowed or only unique values are allowed in the column.

PERSISTENCE.XML

- **The persistence.xml file is a standard configuration file used to configure a given JPA Persistence Unit.**

- **The Persistence Unit defines all the metadata required to bootstrap an EntityManagerFactory like entity mappings, Driver and database details like DB name , username and password as well as JPA provider configuration properties.**

- **The persistence.xml file can also be used to configure automatic database and table creation , caching etc.**

```
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd (xsi:schemaLocation)
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
  http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
    version="2.1">

    <persistence-unit name="dev">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
        <shared-cache-mode>ENABLE_SELECTIVE</shared-cache-mode>      <!-- for caching -->
        <properties>
            <property name="javax.persistence.jdbc.driver"
                value="com.mysql.cj.jdbc.Driver" />
            <property name="javax.persistence.jdbc.url"
                value="jdbc:mysql://localhost:3306/company?createDatabaseIfNotExist=true" />
            <property name="javax.persistence.jdbc.user" value="root" />
            <property name="javax.persistence.jdbc.password"
                value="root" />
            <property name="hibernate.show_sql" value="true" />
            <property name="hibernate.hbm2ddl.auto" value="update" />
            <property name="hibernate.dialect"
                value="org.hibernate.dialect.MySQL8Dialect" />

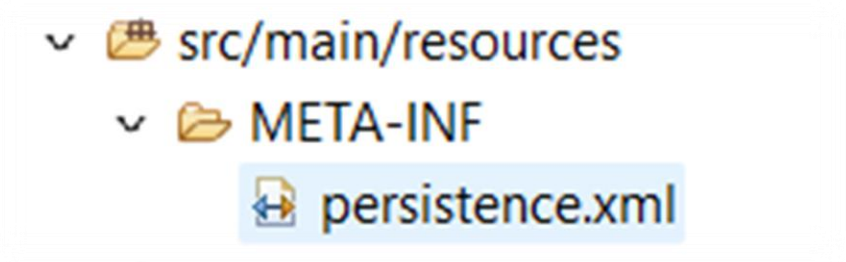
            <!-- for second level caching -->
            <property name="hibernate.cache.use_second_level_cache"
                value="true"></property>
            <property name="hibernate.cache.region.factory_class"
                value="org.hibernate.cache.ehcache.EhCacheRegionFactory"></property>

        </properties>
    </persistence-unit>
</persistence>
```

Persistence.xml file

Persistence.xml file location

- **The persistence.xml file should be located inside META-INF folder which should reside in resources folder , like :**



- **The persistence unit element is used to define a unique name. This name is later used to reference the persistence.xml file and the properties in it.**

- **The provider XML element defines the fully-qualified name of the class(HibernatePersistenceProvider) which implements the JPA PersistenceProvider interface.**

The properties element allows you to define following properties :

- **To specify the Driver to be loaded and registered :**

```
<property name="javax.persistence.jdbc.driver"
                    value="com.mysql.cj.jdbc.Driver" />
```

- **To specify the Url , database name and whether the database should be auto generated.**

```
<property name="javax.persistence.jdbc.url"
                    value="jdbc:mysql://localhost:3306/DataBase_Name" />
```

- **To specify username and password of the database :**

```
<property name="javax.persistence.jdbc.user"
                        value="UserName" />
<property name="javax.persistence.jdbc.password"
                        value="PassWord" />
```

- **To specify whether SQL queries executed internally should be displayed on console or not :**

```
<property name="hibernate.show_sql" value="true" />
```

- **To specify whether table should be generated automatically. If value = create , each time new table will be created which overrides the older one. If value = update , new table will be created only if table doesn't exist.**

```
<property name="hibernate.hbm2ddl.auto" value="update" />
```

- **To specify the hibernate dialect :**

```
<property name="hibernate.dialect"
                value="org.hibernate.dialect.MySQL8Dialect"/>
```

- **To specify whether second level cache should be enabled or not. And if yes , the which second level cache to enable (here , we have enables ehcache second level cache) :**

```
<property name="hibernate.cache.use_second_level_cache"
                                        value="true"></property>

<property name="hibernate.cache.region.factory_class"
        value="org.hibernate.cache.ehcache.EhCacheRegionFactory">
</property>
```

HQL (Hibernate Query Language) / JPQL (Java Persistence Query Language)

- **HQL / JPQL is an object-oriented query language similar to SQL but works with entity objects, not tables.**
- **Instead of querying database tables and columns, we query entities and their properties(variables).**
- **Hibernate translates HQL into native SQL queries internally.**

Example :

SQL - select * from employee ; // here employee is table name

HQL - select e from Employee e ; // here Employee is the entity class name.

SQL – update employee set salary=67000 where name = 'Ram';

HQL – update Employee e set e.salary =67000 where e.name = 'Ram';

STATIC HQL QUERIES

- **Written once and do not change during runtime.**
- **Easy to read and maintain.**
- **Good for fixed operations like fetching all records or specific criteria.**
- **Example : update Employee e set e.salary =67000 where e.name = 'Ram';**

DYNAMIC HQL QUERIES

- **Built dynamically at runtime based on conditions or user input.**
- **Useful when filtering/searching with variable parameters.**

Parameters in HQL

- **To prevent SQL Injection and make code flexible, we use parameters instead of directly concatenating user inputs.**
- **HQL supports two types of parameters:**

Type	Syntax Example	Description
Positional Parameters	?1, ?2, ?3	Based on position.
Named Parameters	:name	Based on parameter name (recommended).

Positional Parameters

- Represented by ? followed by index / position.
- Order matters — must be set in the same sequence.
- Example :

```
Query query = manager.createQuery("update Employee e set e.salary = 1? where e.name = 2?");

    query.setParameter(1 , 90000);

    query.setParameter(2 , "Sheela");

    Query.executeUpdate();
```

Named Parameters

- **More readable and less error-prone.**
- **Use :parameterName in the HQL query and set value using setParameter().**

- **Example :**

```
Query query = manager.createQuery("select  e from Employee e where e.name = :ename ");
    query.setParameter ( "ename" ,  Rahul );
    List<Employee> list = query.getResultList();
```

MAPPING

- **Mapping in a key feature which allows to establish a relationship between two or more entities.**

- **It is used to define how different classes (called entities) are connected to each other in a relational database.**

- **There are four types of mappings :**
    1. **One-to-One mapping**
    2. **One-to-Many mapping**
    3. **Many-to-One mapping**
    4. **Many-to-Many mapping**

1. **One-to-One Mapping :**

In a one-to-one mapping, each instance of one entity is associated with exactly one instance of another entity.

For example, one Person can have only one Aadhar card.

@OneToOne annotation is used.

2. **One-to-Many Mapping :**

A one-to-many mapping means one instance of an entity is associated with multiple instances of another entity.

For example , one student can be enrolled to many courses.

@OneToMany annotation is used.

3. **Many-To-One Mapping :**

A many-to-one mapping means multiple instance of an entity os associated with only one instance of another entity.

For Example , Many employees work in One Department.

@ManyToOne annotation is used.

4. **Many-to-Many Mapping :**

A many-to-many mapping means multiple instances of one entity are associated with multiple instances of another entity.

For Example, Many Questions can have Many Answers.

@ManyToMany annotation is used.

@JoinColumn :

- **What it does:**

  It used on the owning side.

  It creates the foreign key column in the database

  Used in @OneToOne, @ManyToOne, and @OneToMany (if unidirectional).

- **For Example , Imagine a Passport table where each passport is linked to a Person.**

  The passport mentions the person ID inside it — this is @JoinColumn.

- **When to use:**

  Whenever you're on the owning side of the relationship and want to define the FK.

```java
@Entity
public class Student {
    @Id
    private int id;
    private String name;

    @OneToOne
    @JoinColumn(name = "laptop_id") // foreign key column created in Student table
    private Laptop laptop;
}
```

```java
@Entity
public class Laptop {
    @Id
    private int id;
    private String brand;
}
```

Hibernate will now create a Student table with a foreign key column laptop_id which points to the primary key of Laptop table.

mappedBy :

What it does :

mappedBy is used in bi-directional relationships.

It says, "I'm not the owner of the relationship — the other entity owns it.

It is Used to avoid duplicate or unnecessary join tables.

The owner side is responsible for the foreign key column in the database.

The non-owner side uses mappedBy to point to the owner.

- **For Example , Think of a Teacher and a Class:**

  A Class has many Students.

  But we keep the list of students inside the Class entity.

  That means Class owns the relationship.

Simple Example: One-to-Many with mappedBy

```java
@Entity
public class Teacher {
    @Id
    private int id;
    private String name;

    @OneToMany(mappedBy = "teacher")  // 👉 Non-owner side
    private List<Student> students;
}
```

```java
@Entity
public class Student {
    @Id
    private int id;
    private String name;

    @ManyToOne          // 👉 Owner side
    @JoinColumn(name = "teacher_id") // FK will be created here
    private Teacher teacher;
}
```

- **The Student entity is the owner because it has @JoinColumn.**
- **The Teacher entity is the non-owner, so it uses mappedBy = "teacher" (the variable name in Student class).**
- **This avoids creating extra join tables.**

When to use:

On the non-owning side of a bidirectional relationship .(In the owning side , we use @JoinColumn) to Prevents extra join tables.

CASCADE

In JPA , cascade is used to inform to perform certain operations automatically on associated (related) entities when we perform them on the main entity.

When two tables (entities) are related, cascade helps us automatically perform actions on the child entity when we do something to the parent entity.

For example :

- **Imagine a Doctor and his appointments.**

  When a doctor is removed from the Hospital , all his appointments will be automatically cancelled(removed).

Why do we use cascade?

- **To avoid writing extra code.**
- **If we save/delete/update the parent, and want the child to be saved/deleted/updated automatically, we use cascade.**

- **Types of Cascade (with examples):**

Cascade Type	What it does	Easy Example
`PERSIST`	Saves child when parent is saved	Saving a father will also save the son
`MERGE`	Updates child when parent is updated	Updating father's info updates son's info
`REMOVE`	Deletes child when parent is deleted	Deleting father also deletes the son
`REFRESH`	Reloads child from DB when parent is refreshed	Refresh father, son also refreshed
`DETACH`	Removes child from memory when parent is removed	Father detached, son also detached
`ALL`	Does all above things	Full package: save, update, delete, etc.

```java
@Entity
public class Father {
    @Id
    private int id;
    private String name;

    @OneToOne(cascade = CascadeType.ALL)
    private Son son;
}
```

```java
@Entity
public class Son {
    @Id
    private int id;
    private String name;
}
```

- **Without Cascade: You have to perform operations on both father and son manually.**
- **With Cascade: Only perform operation on father, and the same happens to son automatically.**

CACHING

Caching in hibernate is a technique that allows frequently accessed data to be stored in memory so that it can be quickly retrieved without having to go back to the database every time.

By caching data, Hibernate can reduce the number of database queries it needs to execute, improving performance and reducing the load on the database server.

LEVELS OF CACHING

Hibernate provides two levels of caching :

1. **First-Level Cache: Hibernate uses a session-level (EntityManager) level cache , known as a first-level cache. It is used to store the data that is currently being used by a specific session or EntityManager.**

Whenever an entity is fetched from the database, it is stored in the cache.

If the same entity is requested again within the same session, Hibernate retrieves it from the cache instead of hitting the database.

This can improve the performance of the application by reducing the number of database queries.

- **The first-level cache is session level(entity manager level), which means that the data stored in the cache is available only within the session(entity manager). When the session is closed, the cache is cleared automatically. The cache is also not shared between different sessions(entity managers), which means that each session(entity manager) has its own separate cache.**

- **Hibernate manages the first-level cache automatically, and there is no need to configure it manually. The cache is enabled by default and can't be turned off.**

- **When an entity is fetched from the database, Hibernate checks if it is already present in the cache. If it is, the cached entity is returned, otherwise, Hibernate fetches the entity from the database and stores it in the cache.**

2. Second-Level Cache : **Second-level caching is an optional caching mechanism which is a shared cache across multiple sessions. It is entity manager factory level cache which is shared by all entity managers.**

It is more powerful than first-level caching, which is limited to a single session.

The second level cache stores data that is frequently used across different sessions(entity managers), reducing the number of database queries and improving the overall performance of the application.

To enable Second-level Caching in Hibernate, you need to configure a cache provider in your Hibernate configuration file.

The second-level cache can be configured with various caching providers, such as Ehcache etc.

To enable ehcache second level cache

- **First add the following properties in configuration file i.e..persistence.xml file :**

```
<property name="hibernate.cache.use_second_level_cache"
                                                    value="true"></property>

<property name="hibernate.cache.region.factory_class"
        value="org.hibernate.cache.ehcache.EhCacheRegionFactory"></property>
```

- **Also, hibernate ehcache relocation dependency must be added in pom.xml file .**

```
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-ehcache</artifactId>
    <version>5.6.15.Final</version>
</dependency>
```

- **NOTE : The version of ehcache dependency and hibernate dependency must be same.**